

Name: K RAJ BHARATH K RAJ BHARATH | Reg. No: RA2511003011548

Unit – 5 : Session – 8 : SLO – 1 | Shortest Path — Dijkstra's Algorithm

■ I. Fill in the Blanks

- Dijkstra's algorithm is used to find the SHORTEST path from a source vertex to all other vertices.
- Dijkstra's algorithm cannot handle graphs with NEGATIVE edge weights.
- The primary data structure used to track minimum distances is a PRIORITY QUEUE (min-heap).
- The time complexity of Dijkstra's using a priority queue (min-heap) is $O((V + E) \log V)$.
- Dijkstra's algorithm is a type of GREEDY algorithm.

■ II. True or False

- Dijkstra's algorithm is applicable to both directed and undirected graphs — TRUE.
- Dijkstra's algorithm can be used for negative weight edges — FALSE.
- The shortest path from source to destination always forms a tree structure — TRUE (shortest-path tree).
- Dijkstra's algorithm always visits the closest unvisited vertex next — TRUE.

■ III. Reasoning Questions

- Negative weights: Dijkstra's finalizes a vertex's distance when it is visited; a later negative edge could improve an already-finalized distance, breaking the greedy assumption. (Use Bellman-Ford instead.)
- Difference from Prim's: Prim's computes a MINIMUM SPANNING TREE (minimum total edge weight connecting all vertices); Dijkstra's computes SHORTEST PATH distances from one source to all vertices.
- Real-life application: GPS / Google Maps route finding (road network with travel-time weights); also network routing protocols (OSPF).

■ IV. Match the Following

1. Dijkstra's Algorithm — d) Cannot handle negative weight edges
2. Priority Queue — f) Data structure to pick minimum value
3. Source vertex distance — e) Initialized to 0
4. Graph with negative edge weight — d) (cannot be handled by Dijkstra's; needs Bellman-Ford)
5. Kruskal's Algorithm — c) Computes Minimum Spanning Tree
6. Dijkstra vs Prim's — a) One for shortest path, one for MST

■ V. Trace the Table — Graph: 0-1(2), 0-2(6), 1-3(5), 2-3(8), 3-4(10), 3-5(15), 4-5(6), 4-6(2), 5-6(6)

- Iter | Visited | dist[0] | dist[1] | dist[2] | dist[3] | dist[4] | dist[5] | dist[6]
- 0 | — | 0 | inf | inf | inf | inf | inf | inf
- 1 | 0 | 0 | 2 | 6 | inf | inf | inf | inf
- 2 | 1 | 0 | 2 | 6 | 7 | inf | inf | inf
- 3 | 2 | 0 | 2 | 6 | 7 | inf | inf | inf

- 4 | 3 | 0 2 6 7 17 22 inf
- 5 | 4 | 0 2 6 7 17 22 19
- 6 | 6 | 0 2 6 7 17 22 19
- 7 | 5 | 0 2 6 7 17 22 19 (final)

■ VI. Priority Queue State (vertex : distance)

- Start: { 0:0 }
- After selecting 0: { 1:2, 2:6 }
- After selecting 1: { 2:6, 3:7 }
- After selecting 2: { 3:7 }
- After selecting 3: { 4:17, 5:22 }
- After selecting 4: { 6:19, 5:22 }
- After selecting 6: { 5:22 }
- After selecting 5: { } (done)